Due date: Wednesday 8:50am, Nov. 4, 2026

Any function, library and API in OpenCV is not allowed to use, unless it is mentioned in the question.

1. Noise Reduction (C/C++) (35%)

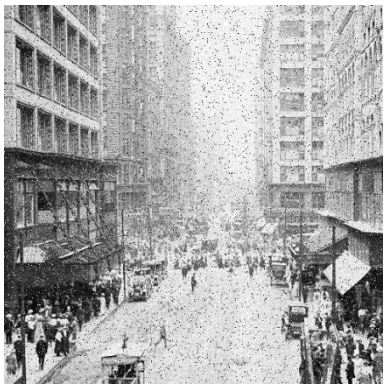
   Given the grayscale image affected by gaussian noise and salt pepper noise, *town500_noisy.raw*, you will apply following method to reduce the noise and compare the results.

   (1) Apply **mean filter(3x3, 5x5) and 5x5 gaussian filter(σ =0.8, 1.5)** to the noisy image. Save the four denoised images. (Figure, 10%)

   (2) Apply a **median filter(3x3, 5x5)** to the noisy image. Save the four denoised images. (Figure, 10%)

   (3) Apply cascade filtering to the noisy image using the following four combinations: **Median → Gaussian, Gaussian → Median, Mean → Gaussian, and Gaussian → Mean**. Use a **3×3 Median Filter**, a **5×5 Mean Filter**, and a **5×5 Gaussian Filter** with **σ = 0.8**. Save the four denoised images. (Figure, 10%)

   (4) Calculate the **PSNR** of results with *town500.raw*. Compare the results and discuss the filtering performance and Reversibility of Serial Connections. (Discussion, 5%)



town500_noisy.raw

2. Iterative Back Projection (IBP) Super-Resolution (C/C++) (20%)

   Given *lena256.raw*, apply Iterative Back Projection (IBP) to reconstruct a 512×512 high-resolution image. Use Bilinear Interpolation to upsample the LR image as the initial HR image. In each iteration, generate a simulated LR image by filtering and downsampling the current HR image, calculate the error map between the original LR image and the simulated LR image, then upsample and filter the error map and add it back to the current HR image.

   (1) Perform IBP reconstruction using 1, 3, 5, and 10 iterations for image. Save the four reconstructed results and calculate the PSNR with *lena512.raw* for each iteration. (Figure, 15%)

   (2) Compare the IBP results with Nearest-Neighbor Interpolation and Bilinear Interpolation. Discuss which method gives better reconstruction quality based on the visual results and PSNR, and explain the advantages and disadvantages of IBP. (Discussion, 5%)

3. Edge Detection (C/C++) (45%)

Apply the following edge detection methods to both *cameraman256.raw* and *cameraman256_noise.raw*. Save the results.

(OpenCV functions may be used for the calculation of Pratt's FOM)

(1) Apply the 4-neighbor and 8-neighbor Laplacian filters to both images. (Figure, 10%)

(2) Apply the 0° Sobel, 90° Sobel, and combined Sobel filters to both images. (Figure, 10%)

(3) Apply the Laplacian of Gaussian (LoG) filter to *cameraman256_noise.raw*. (Figure, 5%)

(4) Binarize all edge-detection results and evaluate their performance using Pratt's Figure of Merit (FOM). Using the edge map of ***cameraman256.raw*** as the ground truth (GT), evaluate and compare the corresponding results from ***cameraman256_noise.raw*** for each method, then complete the table below. Finally, discuss how the choice of threshold affects the FOM, and analyze the performance of each edge detection method. (Result, 10%; Discussion, 10%)



cameraman256.raw



cameraman256_noise.raw

Comparison form

GT / Noise	4-neighbor Laplacian	8-neighbor Laplacian	combined Sobel
4-neighbor Laplacian			
8-neighbor Laplacian			
combined Sobel			
LoG			